UNIVERSIDAD DEL BÍO-BÍO

10 Umbralización RGB

Tutorial Procesamiento de Imagen con webcam

Estudiantes Practicantes:
Javiera Gutiérrez
Javiera Henríquez
Teresa Vidal

Profesor:
Luis Vera

Laboratorio CIMUBB

2023-1

Umbralización de imagen a color

Para umbralizar una imagen a color necesitamos asignarle valores a cada color de su RGB (rojo, verde y azul), para poder resaltar de mejor manera la parte que queremos.

Creamos un nuevo archivo .py y le añadimos las siguientes librerías:

```
10 Umbralización RGB.py ×
10 Umbralización RGB.py > ...
import tkinter as tk
from tkinter import *
from PIL import Image
from PIL import ImageTk
import imutils
import cv2
```

Creamos una ventana usando la librería Tkinter, le asignamos un tamaño, que el usuario no pueda editar el tamaño de la ventana y un título.

```
# Crea ventana, define tamaño y título
ventana = tk.Tk()
ventana.geometry("1070x470")
ventana.resizable(0,0)
ventana.title("Umbralización RGB")
```

A continuación, se crean las funciones para iniciar la webcam y tomar una foto, que vimos en los tutoriales anteriores.

```python
#Funciones cámara web
def camara():
    global capture
    capture = cv2.VideoCapture(0)
    iniciar()

def iniciar():
    global capture
    if capture is not None:
        ret, frame = capture.read()
        if ret == True:
            frame = imutils.resize(frame, width=311)
            frame = imutils.resize(frame, height=241)
            ImagenCamara = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            im = Image.fromarray(ImagenCamara)
            img = ImageTk.PhotoImage(image= im)
            LImagen.configure(image= img)
            LImagen.image = img
            LImagen.after(1,iniciar)
        else:
            LImagen.image = ""
            capture.release()

#Función para tomar una foto
def Capturar():
    global Captura
    camara = capture
    return_value, image = camara.read()
    frame = imutils.resize(image, width=301)
    frame = imutils.resize(frame, height=221)
    Captura = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    im = Image.fromarray(Captura)
    img = ImageTk.PhotoImage(image= im)
    LImagenROI.configure(image= img)
    LImagenROI.image = img
```

Creamos una nueva función llamada **rbg()** en la que, con datos entregados por el usuario, crearemos una máscara usando los máximos y mínimos de cada color, este rango se le asigna a nuestra captura, y desplegamos el resultado en el tercer cuadro gris de nuestra ventana.

```python
def rgb():
    Minimos = (int(SRedI.get()),int(SGreenI.get()),int(SBlueI.get()))
    maximos = (int(SRedD.get()),int(SGreenD.get()),int(SBlueD.get()))
    img_mask = cv2.inRange(Captura, Minimos, maximos)
    img_aux = img_mask
    img_mask = Image.fromarray(img_mask)
    img_mask = ImageTk.PhotoImage(image= img_mask)
    ImagenUmbra.configure(image=img_mask)
    ImagenUmbra.image = img_mask
    _, bin_imagen = cv2.threshold(img_aux, 0, 255, cv2.THRESH_BINARY_INV)
```

Se crean las barras para que el usuario mueva y seleccione los mínimos y máximos de cada color, iniciando en 1 para que no se produzca un error en el programa.

```
#RGB se inicia en 1, ya que si no sale error de división por 0
SRedI = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SRedI.place(x=760,y=300)
SGreenI = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SGreenI.place(x=760, y= 340)
SBlueI = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SBlueI.place(x=760, y= 380)

SRedD = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SRedD.set(255)
SRedD.place(x=900,y=300)
SGreenD = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SGreenD.set(255)
SGreenD.place(x=900, y= 340)
SBlueD = tk.Scale(ventana, from_=1, to=255, orient='horizontal')
SBlueD.set(255)
SBlueD.place(x=900, y= 380)
```

Se le añaden **Label** con la letra inicial representando cada color.

```
#Label
LRed = tk.Label(ventana, text="R")
LRed.place(x=870,y=320,width=21,height=16)
LGreen = tk.Label(ventana, text="G")
LGreen.place(x=870,y=360,width=21,height=16)
LBlue = tk.Label(ventana, text="B")
LBlue.place(x=870,y=400,width=21,height=16)
```

Añadimos tres botones, que accionarán las funciones de iniciar cámara, tomar una foto y umbralizar la imagen.

```
#Botones
BCamara = tk.Button(ventana, text="Iniciar cámara", command=camara)
BCamara.place(x=150,y=330,width=90,height=23)
BCapturar = tk.Button(ventana, text="Tomar foto", command=Capturar)
BCapturar.place(x=500,y=330,width=91,height=23)
Umbra = tk.Button(ventana, text="Umbralizacion", command=rgb)
Umbra.place(x=840,y=430,width=80,height=23)
```

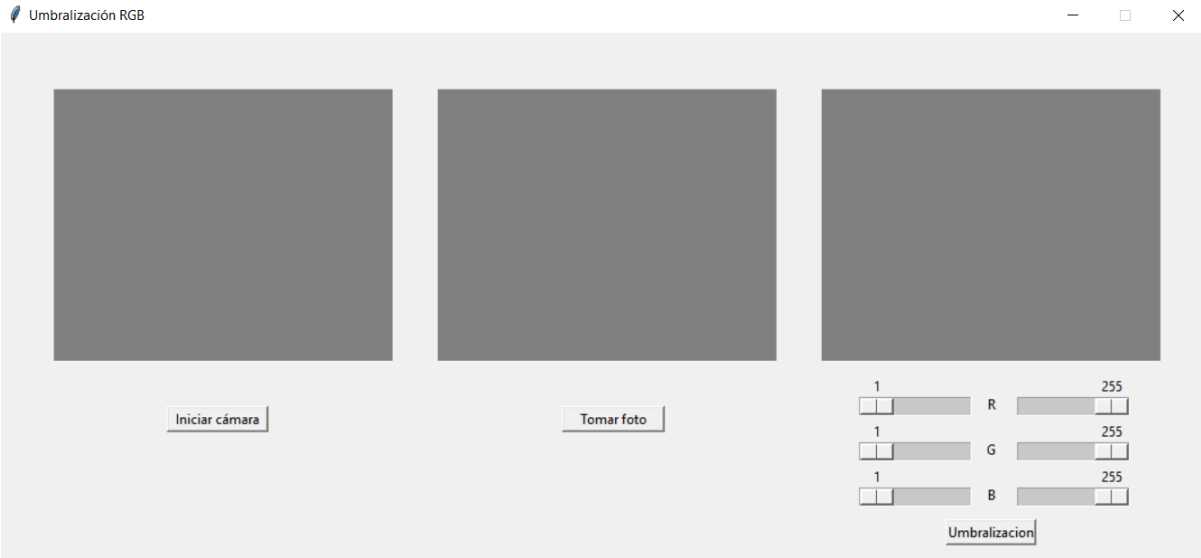
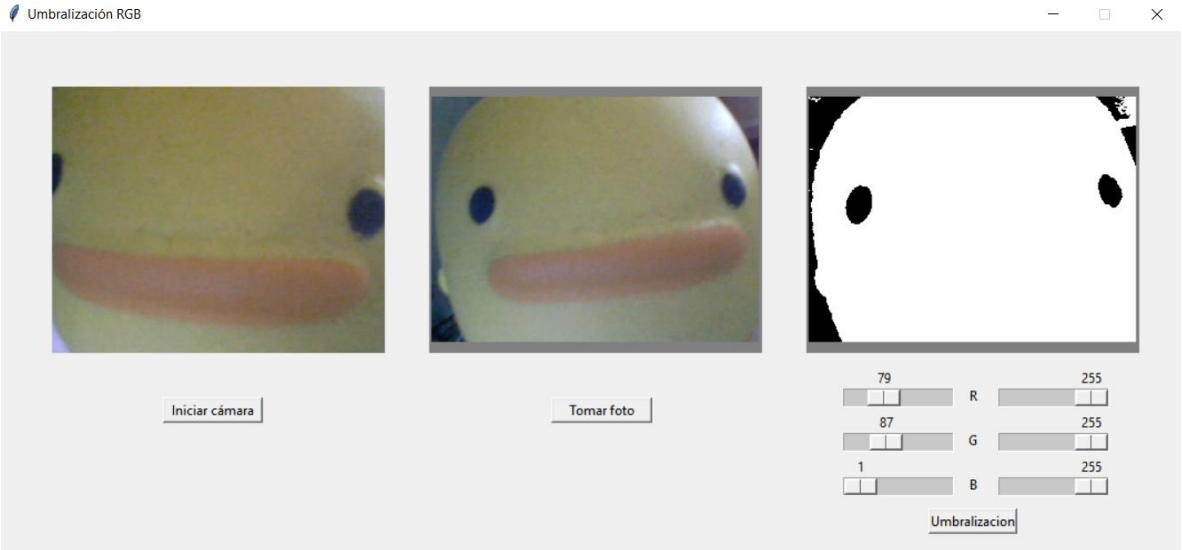
Se crean los tres cuadros grises, donde se desplegará la webcam, la foto tomada y la imagen umbralizada.

Finalmente, cerramos el programa escribiendo **ventana.mainloop()** para que nuestro programa se muestre correctamente.

```
#Cuadros de imagen gris
LImagen = tk.Label(ventana, background="gray")
LImagen.place(x=50,y=50,width=300,height=240)
LImagenROI = tk.Label(ventana, background="gray")
LImagenROI.place(x=390,y=50,width=300,height=240)
ImagenUmbra = tk.Label(ventana, background="gray")
ImagenUmbra.place(x=730,y=50,width=300,height=240)


ventana.mainloop()
```

Una vez compilado y ejecutado, nuestro programa se verá así:



Y al umbralizar una imagen se verá así: